

IFT436 - Devoir #2

À remettre avant le **vendredi 2 octobre à 23h59**.
Tout retard entraînera jusqu'à 33% de pénalité par jour.

Pondération. Ce travail compte pour 6% des points de la session. Si votre note dépasse 100, elle sera rabaisée à 100.

Modalités. Considérez les points suivants.

- Le travail peut être fait seul ou en équipe de deux.
- Le devoir doit être remis en format pdf via turnin. La provenance de votre pdf n'a pas d'importance — ce pdf peut provenir d'un export d'un fichier Word ou d'un fichier texte, d'un scan de vos écrits/dessins, etc.
Il n'y a pas de norme de présentation particulière. Écrivez lisiblement, et assurez-vous seulement que vos noms et CIP apparaissent clairement sur le devoir remis.
- Lorsqu'une complexité en temps est demandée, nous supposons le pire cas, sauf mention contraire.
- Vous pouvez utiliser les techniques de preuves que vous désirez, les indices ne sont que des suggestions.
- Vous pouvez utiliser les résultats démontrés en classe et dans les solutionnaires d'exercices sans avoir à les démontrer. Par contre, si vous utilisez de nouveaux résultats, vous devez les démontrer.

Question 1 : un peu de grand-O (30 points)

Entraînez-vous un peu à classer les fonctions en taux de croissance.

a. Répondez par oui ou par non aux questions suivantes. Vous n'avez pas à justifier vos réponses (ceci sera fait au prochain numéro).

(i) Soient $f(n) = n^4 + 20n^2$ et $g(n) = (n^2/1000 + 1)^2$
Est-ce que $f(n) \in \Theta(g(n))$?

(ii) Soient $f(n) = \log n$ et $g(n) = \log_{10}(n^{436})$
Est-ce que $f(n) \in \Omega(g(n))$?

(iii) Soient $f(n) = (2n)!$ et $g(n) = n!$
Est-ce que $f(n) \in O(g(n))$?

(iv) Soient $f(n) = \log((2n)!)$ et $g(n) = \log(n!)$
Est-ce que $f(n) \in O(g(n))$?

(v) Soient $f(n) = \sum_{i=1}^n 2^i$ et $g(n) = 4^n$
Est-ce que $f(n) \in \Omega(g(n))$?

b. Choisissez **deux sous-questions de la question précédente** et démontrez vos réponses, en utilisant les définitions appropriées de O , Ω et Θ . Vous devez choisir au moins un de (iii), (iv) et (v).

Solution.

(i) Notons avant tout que $g(n) = n^4/1000000 + 2n^2/1000 + 1$.
On montre d'abord que $f(n) \in O(g(n))$. On a

$$\begin{aligned} f(n) &= n^4 + 20n^2 \\ &\leq n^4 + 20n^4 \quad \text{pour } n \geq 1 \\ &\leq 21n^4 \\ &= 21 \cdot 100000(n^4/100000) \\ &\leq 21 \cdot 100000(n^4/100000 + 2n^2/1000 + 1) = c \cdot g(n) \end{aligned}$$

pour tout $n \geq 1$, et donc $f(n) \in O(g(n))$.
On montre ensuite que $f(n) \in \Omega(g(n))$. On a

$$\begin{aligned}
f(n) &= n^4 + 20n^2 \\
&\geq n^4 \\
&= \frac{1}{3}(n^4 + n^4 + n^4) \\
&\geq \frac{1}{3}(n^4/100000 + 2n^4/1000 + n^4) \\
&\geq \frac{1}{3}(n^4/100000 + 2n^2/1000 + 1) = c \cdot g(n)
\end{aligned}$$

pour tout $n \geq 1$, et donc $f(n) \in \Omega(g(n))$.

On déduit que $f(n) \in \Theta(g(n))$.

Bien sûr, il y a plusieurs solutions possibles, l'important est que chaque inégalité doit être vraie. Il était aussi possible de passer par les lois des limites, le ratio des limites devrait donner une constante.

(ii) Oui, en fait $f(n) \in \Theta(g(n))$. Pour montrer que $f(n) \in \Omega(g(n))$, on observe que

$$g(n) = \log_{10}(n^{436}) = 436 \log_{10} n = 436 \log n / \log 10 = c \cdot f(n)$$

où c est une constante. Puisqu'on a seulement des égalités dans cette chaîne, on peut déduire que $g(n) \in \Theta(f(n))$, ce qui implique en particulier que $f(n) \in \Omega(g(n))$.

(iii) Non. Supposons pour fins de contradiction que $(2n)! \in O(n!)$. Alors il existe n_0 et c tels que pour tout $n \geq n_0$, on a $(2n)! \leq cn!$. Donc

$$\begin{aligned}
(2n!) &= (2n)(2n-1)(2n-2) \dots (n+1)(n)(n-1) \dots (2)(1) \\
&= (2n)(2n-1)(2n-2) \dots (n+1) \cdot n! \\
&\leq c \cdot n!
\end{aligned}$$

En divisant par $n!$ sur les deux dernières expressions, ceci implique que $(2n)(2n-1)(2n-2) \dots (n+1) \leq c$ pour tout $n \geq n_0$. Cette conclusion implique que c tend vers l'infini lorsque n grandit, ce qui contredit le fait que c est une constante, donc l'énoncé doit être faux.

(iv) Oui. Dans les exercices, on a vu que $\log(n!) \in \Theta(n \log n)$. En remplaçant

$n!$ par $(2n)!$, on déduit que $\log((2n)!) \in \Theta(2n \log(2n)) = \Theta(n \log n)$, donc les deux fonctions ont le même taux de croissance. J'acceptais une telle réponse. On pouvait aussi y aller dans le détail. On a

$$\begin{aligned} \log((2n)!) &= \sum_{i=1}^{2n} \log(i) \\ &\leq \sum_{i=1}^{2n} \log(n) \\ &\leq 2n \log(2n) \\ &\leq 2n \log(n^2) \\ &= 4n \log n \end{aligned}$$

(il y a clairement un n_0 à partir duquel la dernière inégalité est vraie, sa valeur exacte ne nous intéresse pas). Par ailleurs, les exercices montrent que $\log(n!) \in \Theta(n \log n)$, ce qui veut dire qu'il existe n_1 et c tels que pour tout $n \geq n_1$, on a $\log(n!) \geq cn \log n$. Donc pour tout $n \geq \max(n_0, n_1)$, on a

$$\log(2n!) \leq 4n \log n = \frac{4}{c}(cn \log n) \leq \frac{4}{c} \log(n!)$$

Donc $\log(2n!) \in O(\log(n!))$.

(v) Non. On a

$$f(n) = \sum_{i=1}^n 2^i = \sum_{i=0}^n 2^i - 2^0 = 2^{n+1} - 1 - 1 = 2^{n+1} - 2.$$

Pour fins de contradiction, si on suppose que $2 \cdot 2^n - 2 \geq c \cdot 4^n$ pour une certaine constante c et pour tout $n \geq n_0$. En isolant c , on obtient

$$c \leq 2/2^n - 2/4^n.$$

Mais c approche 0 lorsque n tend vers l'infini, et donc c ne peut pas être une constante (car c devient plus petit que n'importe quelle constante), une contradiction.

Question 2 : le tri sélection (15 points)

Le *tri sélection* implémente la stratégie suivante pour trier un tableau T . On initialise d'abord un tableau de sortie S vide. Puis on répète jusqu'à ce que T soit vide: on trouve le minimum m de T , on enlève m de T et on ajoute m à la fin de S .

Donnez la complexité en temps de cet algorithme dans le pire cas avec la notation Θ . Justifiez votre réponse.

Remarque. À vous de choisir la structure de données utilisée pour représenter T et S . Vous pouvez utiliser sans preuve les complexités connues pour les structures de données standard.

Solution.

Pour voir que l'algorithme est en temps $O(n^2)$, on observe qu'il y a une première boucle qui supprime un élément à chaque itération, et donc cette boucle fera au plus n itérations. Il faut regarder combien de temps prend un tour de cette boucle.

Trouver le minimum peut facilement se faire en temps $O(n)$. Si on suppose que T et S sont des vecteurs (des tableaux), retirer un élément de T et ajouter dans S prennent un temps $O(n)$ chacun. Donc, un tour de boucle prend un temps $O(n + n) = O(n)$, et puisqu'on fait n tours au plus, le temps est $O(n^2)$.

Pour voir que l'algorithme est en temps $\Omega(n^2)$, on note que l'algorithme fait *toujours* n tours de la première boucle, puis qu'à chaque tour, il faut parcourir les éléments de T pour trouver le minimum. À l'itération $i \in \{1, \dots, n\}$, il reste $n - i + 1$ éléments dans T . Donc sur toute entrée entrée, le nombre d'instructions sera au moins

$$n + (n - 1) + (n - 2) + \dots + 2 + 1 = n(n + 1)/2 \in \Omega(n^2).$$

Les éléments recherchés dans cette analyse:

- pour Ω considérer le fait que le nombre d'itérations de la seconde boucle diminue pendant l'exécution, mais que ça reste quand même $\Omega(n^2)$;
- argumenter sur O et Ω . Il faut bien clarifier que dans le cas O , on s'intéresse à borner le nombre max d'instructions, et dans le cas Ω on veut donner une borne inférieure.
- si vous avez une instruction du style *T.erase*, tenir compte du fait que ce n'est PAS une opération $O(1)$, à moins de spécifier une façon (retirer un élément d'un tableau demande de décaler plusieurs valeurs)

Question 3 : détection de collision 1D (30 points)

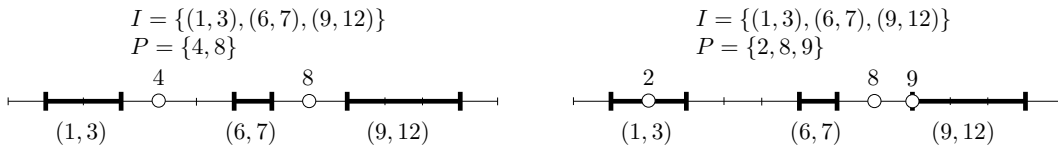
Vous voulez prendre rendez-vous avec un professeur qui a une liste d'intervalles de disponibilités. De plus, vous avez une liste de moments précis où vous pourriez prendre ce rendez-vous. Vous voulez savoir s'il y a un de ces moments qui entre dans ces disponibilités. Ce genre de problème survient dans de multiples contextes sous d'autres formes. On va donc l'abstraire en le formalisant comme suit.

Problème de la collision 1D

Entrée: un ensemble d'entiers $P = \{p_1, p_2, \dots, p_n\}$ représentant les points 1D, et un ensemble de paires d'entiers $I = \{(a_1, b_1), (a_2, b_2), \dots, (a_m, b_m)\}$ représentant des intervalles de segments.

Sortie: une paire $(p_i, (a_j, b_j))$, où $p_i \in P$ et $(a_j, b_j) \in I$ tels que $a_j \leq p_i \leq b_j$, ou bien *null* s'il n'existe pas de telle paire point/intervalle.

La figure ci-dessous donne une représentation visuelle du problème. Dans l'exemple de gauche, on retournerait *null*. Dans l'exemple de droite, vous pouvez retourner soit $(2, (1, 3))$ ou bien $(9, (9, 12))$. Donc on ne veut qu'une seule paire point/intervalle.



Répondez aux questions suivantes:

- L'algorithme naïf fera la chose suivante: pour chaque $p \in P$ et pour chaque $(a, b) \in I$, vérifier si p est contenu dans l'intervalle entre a et b (inclusivement), en retournant p et (a, b) si c'est le cas, et en retournant *null* si aucun point/intervalle n'a été trouvé. Quelle serait la complexité en temps de l'algorithme naïf, en terme de $|P|$ et $|I|$ en notation O ? Justifiez votre réponse.

Solution.

J'acceptais une réponse aussi simple que:

La complexité sera de $O(|P||I|)$ car on va tester chaque combinaison de point et d'intervalle (et tester chaque combinaison peut se faire en temps constant).

-
- b. Supposons que $|P| = |I|$. Donnez le pseudo-code d'un algorithme pour ce problème qui atteint une meilleure complexité que l'algorithme naïf dans ce cas. Donnez la complexité O de votre algorithme avec justification. Vous n'avez pas à justifier que votre code est correct (mais il doit l'être).
-

Solution.

```
fonction collision1D( $P, I$ )
    Trier  $P$  en ordre croissant
    Trier  $I$  en ordre croissant, selon la valeur min de l'intervalle
     $p = 1$  // Itérateur sur  $P$ 
     $i = 1$  // Itérateur sur  $I$ 
    tant que  $p \leq |P|$  et  $i \leq |I|$  faire
        si  $P[p] < I[i].min$  alors
             $p++$ 
        sinon si  $P[p] > I[i].max$  alors
             $i++$ 
        sinon
            return ( $P[p], I[i]$ )
    fin
fin
return  $nil$ 
```

Pour l'analyse de la complexité, posons $n = |P| = |I|$. Les deux tris prennent un temps $O(n \log n)$ si on utilise par exemple le tri fusion. Ensuite, l'itération prend un temps $O(|P| + |I|) = O(n)$ et donc au total, la complexité est $O(n \log n + n) = O(n \log n)$.

Voici la raison pour laquelle je voulais qu'on suppose que $|P| = |I|$. Cet algo prend en fait un temps $O(|P| \log |P| + |I| \log |I|)$. Supposons que $|I| = 2^{|P|}$. Cet algo devient $O(|I| \log 2^{|P|}) = O(|I||P|)$ et finalement ce n'est pas mieux que l'algo naïf. Bien sûr, ceci est un cas extrême mais il reste que si $|I|$ et $|P|$ sont exponentiellement différents, l'algo naïf peut être mieux.

Argumentons que cet algorithme fonctionne, même si $|P| \neq |I|$ (**vous n'aviez pas à le faire, je le mets pour des raisons pédagogiques**).

On procède par induction sur $|P|+|I|$. Si $|P|+|I| = 1$, c'est trivial. Pour l'induction, on suppose que l'algorithme est correct pour toute instance (P', I') telle que $|P'| + |I'| < |P| + |I|$. Lorsque $p = 1$ et $i = 1$, au tout début, on a que $P[p]$ est le point le plus “à gauche” et $I[i]$ un intervalle dont la minimum est le plus “à gauche” (notez qu'il peut y avoir plusieurs intervalles avec le même minimum).

Si $P[p] < I[i].min$, il est impossible que p intersecte avec un intervalle car il est à gauche de tous les intervalles. Donc on peut l'éliminer, ce qu'on fait en effectuant $p++$. Ceci en revient à résoudre le problème sur $P[2..|P|]$ et I , et dans ce cas l'algorithme est correct, par induction. Si $P[p] > I[i].max$, il est impossible que I intersecte avec un point car il est à gauche de tous les points. Par le même argument, on peut éliminer I et l'algorithme est correct, par induction. Un seul cas n'a pas été considéré: si $P[p] < I[i].min$ est faux et $P[p] > I[i].max$ est faux. Mais alors on a $P[p] \geq I[i].min$ et $P[p] \leq I[i].max$. Dans ce cas, $P[p]$ est contenu dans $I[i]$ et on peut les retourner.

Question 4 : détection de cycles (25 points)

Cette question concerne les graphes non-orientés.

- a. Donnez le pseudo-code d'un algorithme qui, étant donné un graphe $G = (V, E)$, détermine si G contient un cycle et, si c'est le cas, retourne les sommets d'un tel cycle (dans l'ordre qui forme le cycle).

Vous n'avez pas à justifier la complexité de votre algorithme, mais elle doit être aussi basse que possible.

- b. Un *unicycle* est un graphe dans lequel il est possible de retirer une arête pour obtenir un arbre. En d'autres termes, $G = (V, E)$ est un unicycle si $\exists e \in E(G) \cdot (G - e \text{ est un arbre})$, où $G - e = (V, E \setminus \{e\})$.

Dans vos mots¹, expliquez comment concevoir un algorithme qui détermine si un graphe est un unicycle. Donnez ensuite la complexité O en temps de cette algorithme.

Solution.

(a) On peut faire un fouille en largeur modifiée, l'idée est que dès qu'un sommet a un voisin déjà vu autre que son parent, on peut boucler le cycle. Il faut tout de même gérer le cas où le graphe n'est pas connexe! Pour cette raison, je démarre un fouille en largeur à partir de chaque sommet — en m'assurant que si un sommet a déjà été rencontré par une fouille précédente je n'ai pas à faire cette fouille.

Une solution possible était de démarrer une nouvelle fouille en largeur à partir de chaque sommet. Ça fonctionne, mais ça prendra un temps $O(n(n + m))$, ce qui est sous-optimal et pénalisable.

Reconstruire un cycle concret n'est pas si trivial. La procédure `reconstruire_cycle` fait ça. Étant donné u, v qui forment une arête, l'idée est de trouver le dernier ancêtre commun z de u et v dans l'arbre de fouille en largeur. Ensuite, le cycle est forme du chemin de z à v , suivi du chemin de u à z .

Le tout prend un temps $O(n + m)$. Ce n'était pas à justifier dans votre réponse, mais voici tout de même les détails. Ceci est parce que la fouille en largeur prend un temps $O(n + m)$, pour les mêmes arguments que ceux vus en classe. La procédure `reconstruire_cycle` est appelée une seule fois et prend un temps $O(n)$: stocker les ancêtres de u se fait en temps $O(n)$ via un parcours de l'arbre

¹Ceci veut dire que vous pouvez donner du pseudo-code, mais il est aussi acceptable d'expliquer votre algorithme en phrases, en énonçant seulement les étapes importantes.

de fouille, puis parcourir les ancêtres de v pour trouver l'ancêtre prend aussi un temps $O(n)$ (si on suppose une table de hachage pour les opérations sur les ensembles en temps constant). La reconstruction du cycle se fait aussi en temps $O(n)$ via deux autres parcours de l'arbre.

```

fonction trouverCycle( $G$ )
    visite  $\leftarrow$  dict()
    parent  $\leftarrow$  dict()
    pour chaque  $s \in V(G)$  faire
        si  $s \notin \textit{visite}$  alors
            parent[ $s$ ]  $\leftarrow$  null
            file  $\leftarrow$  [ $s$ ]
            visite[ $s$ ]  $\leftarrow$  vrai
            tant que file n'est pas vide faire
                 $v \leftarrow \textit{file.pop}()$ 
                pour chaque  $u \in N_G(v)$  faire
                    si  $u \notin \textit{visite}$  alors
                        visite[ $u$ ]  $\leftarrow$  vrai
                        parent[ $u$ ]  $\leftarrow$   $v$ 
                        file.push( $u$ )
                    sinon si  $u \neq \textit{parent}[v]$  alors
                        return reconstruire_cycle( $G, u, v, \textit{parent}$ )
            fin
        fin
    fin
    retourner null

```

```

fonction reconstruire_cycle( $G, u, v, \text{parent}$ )
    ancetres_u = set()
    cur = u
    tant que cur  $\neq$  null faire
        ancetres_u.insert(cur)
        cur = parent[cur]
    fin
    ancetre_commun = null
    cur = v
    C = []
    tant que cur  $\neq$  null faire
        C.prepend(cur) //ajouter en début
        si cur  $\in$  ancetres_u alors
            ancetre_commun = cur
            break
        cur = parent[cur]
    fin
    cur = u
    tant que cur  $\neq$  ancetre_commun faire
        C.append(cur)
        cur = parent[cur]
    fin

```

(b) On utilise l'algorithme de la question précédente pour trouver un cycle C (s'il n'y en n'a pas, la réponse est *non*). Ceci prend un temps $O(n + m)$. Ensuite, on retire une arête $e = uv$ arbitraire de C . Avec une liste d'adjacence, ceci peut se faire en temps $O(n + m)$. Puis on vérifie en temps $O(n + m)$ si le graphe résultant est un arbre, en vérifiant s'il reste $n - 1$ arêtes et si ce graphe est connexe, tel que vu en classe. Au total, ceci prend un temps $O(n + m)$. Vous n'aviez pas à argumenter que ceci est correct, mais pour le voir, il suffit d'observer que si un graphe est connexe, retirer une arête d'un cycle ne le déconnecte pas (il existera toujours un chemin alternatif entre deux sommets). Donc si le graphe est unicycle, retirer toute arête d'un cycle donnera un arbre.
